

```
{
    $response = $this->get('/');
    $response->assertStatus(200);
}
```

In the above code, you can see that the `get()` method accepts a GET request in the form of a slash. The `assertStatus()` method accepts the HTTP status code as an argument. The Laravel also provides a wide range of assertions using which you can inspect the response headers, content and JSON structure. You can even customise request headers with the help of `withHeaders()` method as shown in the code snippet below. This is to be done before the request is delivered to the application, which enables you to add custom headers.

```
$response = $this->withHeaders([
    'X-Header' => 'Value',
])->json('POST', '/user', ['name' => 'Paul']);
```

Talking about the session and authentication, Laravel provides several helper methods such as `withSession()` and `actingAs()` methods. The `withSession()` method will be useful if you would like the application to load the session with data before initiating a request.

The `actingAs()` method provides a simple way to authenticate a specific user as the active user and for this purpose you can use a model factory. The method accepts one argument, which will be that of the user variable as listed below.

```
$user = factory(User::class)->create();
$response = $this->actingAs($user)
```

The `actingAs()` method also accepts a guard name as the second argument, which will denote whether the guard should be used to authenticate the specific user.

```
$this->actingAs($user, 'api')
```

To test JSON APIs and their responses, the Laravel framework ships with several helpers such as `json()`, `get()`, `post()`, `patch()`, `put()`, and `delete()` methods. You have to make use of these methods to issue requests with several HTTP verbs in addition to the ability to pass data and headers. The `assertJson()` method is used to convert the response to an array. The method also makes use of `PHPUnit::assertArraySubset` to verify whether the array in question exists within the JSON response. The `assertExactJson()` method helps you to find out whether the array is an exact match for the JSON delivered by the application. It should be used in con-